

Greedy Heuristics for the Power Dominating Set (PDS) Problem

PMU Placement for Full Observability of Power Grids

Sadia Tabassum (1905091)

Md. Azizul Haque Nadim (1905059)

Supervised by

Abu Wasif

Associate Professor, Department of CSE, BUET

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

June 2026

Outline

- 1 Motivation
- 2 Problem Definition
- 3 Complexity and Solution Approaches
- 4 Our Approach
- 5 Experimental Results
- 6 Conclusion

Why Monitor a Power Grid?

- Modern power grids must be monitored **in real time**: voltages and phase angles at every bus.

Why Monitor a Power Grid?

- Modern power grids must be monitored **in real time**: voltages and phase angles at every bus.
- Undetected disturbances cascade — e.g. the 2003 North-East US blackout affected ~ 50 million people.

Why Monitor a Power Grid?

- Modern power grids must be monitored **in real time**: voltages and phase angles at every bus.
- Undetected disturbances cascade — e.g. the 2003 North-East US blackout affected ~ 50 million people.
- **Phasor Measurement Units (PMUs)** measure the voltage phasor at a bus *and* the current phasors on all incident lines.

Why Monitor a Power Grid?

- Modern power grids must be monitored **in real time**: voltages and phase angles at every bus.
- Undetected disturbances cascade — e.g. the 2003 North-East US blackout affected ~ 50 million people.
- **Phasor Measurement Units (PMUs)** measure the voltage phasor at a bus *and* the current phasors on all incident lines.
- PMUs are expensive (device + installation + communication infrastructure).

Why Monitor a Power Grid?

- Modern power grids must be monitored **in real time**: voltages and phase angles at every bus.
- Undetected disturbances cascade — e.g. the 2003 North-East US blackout affected ~ 50 million people.
- **Phasor Measurement Units (PMUs)** measure the voltage phasor at a bus *and* the current phasors on all incident lines.
- PMUs are expensive (device + installation + communication infrastructure).

The optimization question

What is the minimum number of PMUs (and where to place them) so that the entire grid is observable?

Why Monitor a Power Grid?

- Modern power grids must be monitored **in real time**: voltages and phase angles at every bus.
- Undetected disturbances cascade — e.g. the 2003 North-East US blackout affected ~ 50 million people.
- **Phasor Measurement Units (PMUs)** measure the voltage phasor at a bus *and* the current phasors on all incident lines.
- PMUs are expensive (device + installation + communication infrastructure).

The optimization question

What is the minimum number of PMUs (and where to place them) so that the entire grid is observable?

- Placement at every bus is wasteful.

Why Monitor a Power Grid?

- Modern power grids must be monitored **in real time**: voltages and phase angles at every bus.
- Undetected disturbances cascade — e.g. the 2003 North-East US blackout affected ~ 50 million people.
- **Phasor Measurement Units (PMUs)** measure the voltage phasor at a bus *and* the current phasors on all incident lines.
- PMUs are expensive (device + installation + communication infrastructure).

The optimization question

What is the minimum number of PMUs (and where to place them) so that the entire grid is observable?

- Placement at every bus is wasteful.
- Kirchhoff's and Ohm's laws let observed quantities **propagate**: some buses are inferred for free.

Origin of the Problem

- Electrical engineers studied PMU placement since the 1990s (observability analysis; Brueni and Heath, 2005).

Origin of the Problem

- Electrical engineers studied PMU placement since the 1990s (observability analysis; Brueni and Heath, 2005).
- **Haynes et al. (2002)** formalized it as a graph problem: the **Power Dominating Set (PDS)** problem.

Origin of the Problem

- Electrical engineers studied PMU placement since the 1990s (observability analysis; Brueni and Heath, 2005).
- **Haynes et al. (2002)** formalized it as a graph problem: the **Power Dominating Set (PDS)** problem.
- PDS generalizes the classical DOMINATING SET problem by adding a *propagation (zero-forcing) rule* derived from Ohm's law.

Origin of the Problem

- Electrical engineers studied PMU placement since the 1990s (observability analysis; Brueni and Heath, 2005).
- **Haynes et al. (2002)** formalized it as a graph problem: the **Power Dominating Set (PDS)** problem.
- PDS generalizes the classical DOMINATING SET problem by adding a *propagation (zero-forcing) rule* derived from Ohm's law.

Practical applications

- State estimation and wide-area monitoring of transmission grids
- Fault detection / blackout prevention
- Related models: zero forcing in quantum control, graph searching, infection spreading

The Power Dominating Set Problem

Given a graph $G = (V, E)$, a set $S \subseteq V$ (the PMU locations) **observes** vertices by exhaustively applying two rules:

Rule 1 — Domination rule

Every vertex in S and every neighbor of a vertex in S is observed: $N[S]$ is observed.

The Power Dominating Set Problem

Given a graph $G = (V, E)$, a set $S \subseteq V$ (the PMU locations) **observes** vertices by exhaustively applying two rules:

Rule 1 — Domination rule

Every vertex in S and every neighbor of a vertex in S is observed: $N[S]$ is observed.

Rule 2 — Propagation rule

If an *observed* vertex v has exactly **one unobserved neighbor** u , then u becomes observed. (repeat until no rule applies)

The Power Dominating Set Problem

Given a graph $G = (V, E)$, a set $S \subseteq V$ (the PMU locations) **observes** vertices by exhaustively applying two rules:

Rule 1 — Domination rule

Every vertex in S and every neighbor of a vertex in S is observed: $N[S]$ is observed.

Rule 2 — Propagation rule

If an *observed* vertex v has exactly **one unobserved neighbor** u , then u becomes observed. (repeat until no rule applies)

S is a **power dominating set** if, after exhaustive application, *all* of V is observed.

The Power Dominating Set Problem

Given a graph $G = (V, E)$, a set $S \subseteq V$ (the PMU locations) **observes** vertices by exhaustively applying two rules:

Rule 1 — Domination rule

Every vertex in S and every neighbor of a vertex in S is observed: $N[S]$ is observed.

Rule 2 — Propagation rule

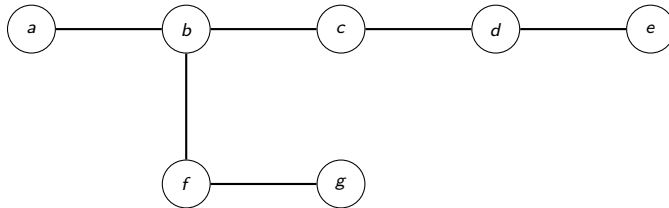
If an *observed* vertex v has exactly **one unobserved neighbor** u , then u becomes observed. (repeat until no rule applies)

S is a **power dominating set** if, after exhaustive application, *all* of V is observed.

Objective

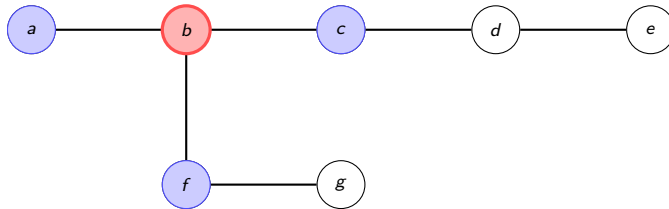
Find a power dominating set of minimum size $\gamma_P(G)$ — the *power domination number*.

Example: One PMU Observes the Whole Graph



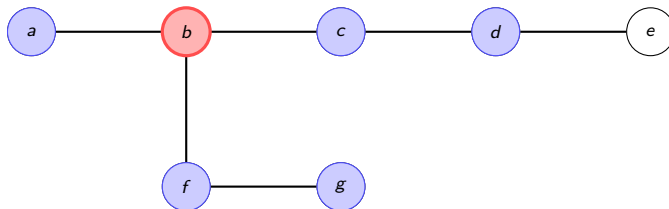
Initially no vertex is observed. Place a single PMU at b .

Example: One PMU Observes the Whole Graph



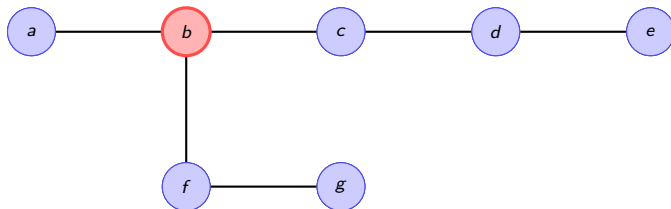
Domination rule: PMU at b observes b and its neighbors $\{a, c, f\}$.

Example: One PMU Observes the Whole Graph



Propagation: c has exactly one unobserved neighbor $d \Rightarrow d$ observed.
Similarly f propagates to g .

Example: One PMU Observes the Whole Graph



Propagation again: d propagates to e . All vertices observed $\Rightarrow \{b\}$ is a PDS, so $\gamma_P(G) = 1$.
(An ordinary dominating set of this graph needs 3 vertices — propagation is what makes PDS different and powerful.)

How Hard is PDS?

- **NP-hard** (Haynes et al., 2002), even restricted to bipartite and chordal graphs; also hard on planar graphs.

How Hard is PDS?

- **NP-hard** (Haynes et al., 2002), even restricted to bipartite and chordal graphs; also hard on planar graphs.
- Hard to approximate (Aazami, 2008).

How Hard is PDS?

- **NP-hard** (Haynes et al., 2002), even restricted to bipartite and chordal graphs; also hard on planar graphs.
- Hard to approximate (Aazami, 2008).
- Polynomial only on restricted classes: trees, bounded treewidth, etc.

How Hard is PDS?

- **NP-hard** (Haynes et al., 2002), even restricted to bipartite and chordal graphs; also hard on planar graphs.
- Hard to approximate (Aazami, 2008).
- Polynomial only on restricted classes: trees, bounded treewidth, etc.

Consequence

For real grid instances (up to thousands of buses) we must choose between *exact but expensive*, *approximate with guarantees*, or *fast heuristics*.

Landscape of Solution Approaches

Exact algorithms

- MILP formulations
- Exact exponential-time algorithms (Binkele-Raible and Fernau, 2012)
- Implicit hitting set + reduction rules (Bläsius and Göttlicher, 2025)

Optimal, but exponential worst case

Landscape of Solution Approaches

Exact algorithms

- MILP formulations
- Exact exponential-time algorithms (Binkele-Raible and Fernau, 2012)
- Implicit hitting set + reduction rules (Bläsius and Göttlicher, 2025)

Optimal, but exponential worst case

Approximation

- Hardness threshold $2^{\log^{1-\epsilon} n}$ — in contrast to only logarithmic hardness for ordinary DOMINATING SET (Aazami and Stilp)
- $O(\sqrt{n})$ -approximation for planar graphs; their methods cannot improve on this guarantee
- Same hardness even on directed acyclic graphs

Strong inapproximability in general; guarantees only for special classes

Landscape of Solution Approaches

Exact algorithms

- MILP formulations
- Exact exponential-time algorithms (Binkele-Raible and Fernau, 2012)
- Implicit hitting set + reduction rules (Bläsius and Göttlicher, 2025)

Optimal, but exponential worst case

Approximation

- Hardness threshold $2^{\log^{1-\epsilon} n}$ — in contrast to only logarithmic hardness for ordinary DOMINATING SET (Aazami and Stilp)
- $O(\sqrt{n})$ -approximation for planar graphs; their methods cannot improve on this guarantee
- Same hardness even on directed acyclic graphs

Strong inapproximability in general; guarantees only for special classes

Heuristics / metaheuristics

- GRASP (Jovanovic and Voss, 2020)
- Multiparent path relinking (MPR), embedded as a post-processing step in GRASP (Laguna, 2025)

Fast, no guarantee — our focus

Our Contribution

- ① A family of **five greedy construction heuristics (H1–H5)** for PDS, built on a single fast $O(m)$ observation engine, using progressively more structural information.

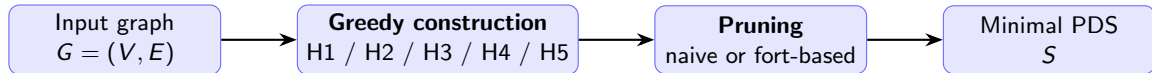
Our Contribution

- ① A family of **five greedy construction heuristics (H1–H5)** for PDS, built on a single fast $O(m)$ observation engine, using progressively more structural information.
- ② A **post-processing pruning stage** that removes redundant PMUs, in two variants: a naive pruner and a **fort-based pruner**.

Our Contribution

- 1 A family of **five greedy construction heuristics (H1–H5)** for PDS, built on a single fast $O(m)$ observation engine, using progressively more structural information.
- 2 A **post-processing pruning stage** that removes redundant PMUs, in two variants: a naive pruner and a **fort-based pruner**.
- 3 A **broad empirical benchmark** over a variety of benchmark datasets (BCSPWR, bus/admittance, IEEE, PEGASE, RTE), with instances ranging from 14 to 9,241 buses, evaluated against *certified optima* and the literature baseline.

Pipeline Overview



Building block: a fast observation engine

- All heuristics repeatedly ask: *“how much of the graph does the current set observe?”*
- The engine applies the domination and propagation rules until no new vertex is observed, updating per-vertex counters of unobserved neighbors incrementally.
- Each edge is processed $O(1)$ times $\Rightarrow O(m)$ **per closure**.

The Greedy Family H1–H5

Generic scheme: start with $S = \emptyset$; while not all observed, add the vertex with the best **score**; recompute observation closure.

Heuristic	Selection rule
H1: Degree-based greedy	selects the highest static-degree vertex

The Greedy Family H1–H5

Generic scheme: start with $S = \emptyset$; while not all observed, add the vertex with the best **score**; recompute observation closure.

Heuristic	Selection rule
H1: Degree-based greedy	selects the highest static-degree vertex
H2: Effective-degree-based greedy	selects the vertex with the most currently unobserved neighbors, recomputed at each step

The Greedy Family H1–H5

Generic scheme: start with $S = \emptyset$; while not all observed, add the vertex with the best **score**; recompute observation closure.

Heuristic	Selection rule
H1: Degree-based greedy	selects the highest static-degree vertex
H2: Effective-degree-based greedy	selects the vertex with the most currently unobserved neighbors, recomputed at each step
H3: Multi-hop greedy	scores vertices by a weighted count of neighbors at distances one, two, and three

The Greedy Family H1–H5

Generic scheme: start with $S = \emptyset$; while not all observed, add the vertex with the best **score**; recompute observation closure.

Heuristic	Selection rule
H1: Degree-based greedy	selects the highest static-degree vertex
H2: Effective-degree-based greedy	selects the vertex with the most currently unobserved neighbors, recomputed at each step
H3: Multi-hop greedy	scores vertices by a weighted count of neighbors at distances one, two, and three
H4: Effective multi-hop greedy	restricts the multi-hop score to unobserved vertices

The Greedy Family H1–H5

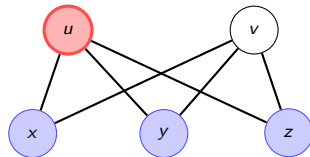
Generic scheme: start with $S = \emptyset$; while not all observed, add the vertex with the best **score**; recompute observation closure.

Heuristic	Selection rule
H1: Degree-based greedy	selects the highest static-degree vertex
H2: Effective-degree-based greedy	selects the vertex with the most currently unobserved neighbors, recomputed at each step
H3: Multi-hop greedy	scores vertices by a weighted count of neighbors at distances one, two, and three
H4: Effective multi-hop greedy	restricts the multi-hop score to unobserved vertices
H5: Near-twin-guided greedy	two-phase: first selects near-twin vertices by degree, then completes the solution with degree-based greedy

Prior Idea 1: Twins

Definition

Two vertices u, v are **twins** if they have the *same neighborhood*: $N(u) = N(v)$.



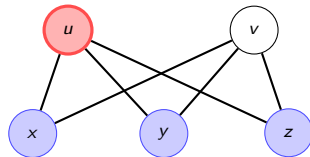
u and v are twins: $N(u) = N(v) = \{x, y, z\}$.
A PMU at u dominates everything v would.

Prior Idea 1: Twins

Definition

Two vertices u, v are **twins** if they have the *same neighborhood*: $N(u) = N(v)$.

- Twins are interchangeable for observation: whatever one of them can dominate, the other can too.



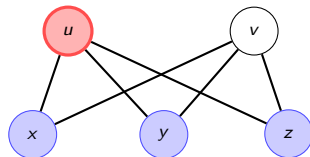
u and v are twins: $N(u) = N(v) = \{x, y, z\}$.
A PMU at u dominates everything v would.

Prior Idea 1: Twins

Definition

Two vertices u, v are **twins** if they have the *same neighborhood*: $N(u) = N(v)$.

- Twins are interchangeable for observation: whatever one of them can dominate, the other can too.
- So placing PMUs on both members of a twin pair is wasteful — one of them suffices.

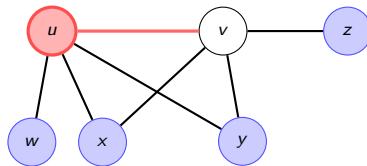


u and v are twins: $N(u) = N(v) = \{x, y, z\}$.
A PMU at u dominates everything v would.

Prior Idea 1 (cont.): Near-Twins

Definition

Two vertices u, v are **near-twins** if they are *adjacent* and their neighborhoods are *nearly identical* (differ in only a few vertices).



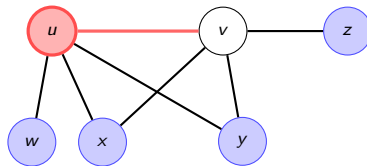
u, v are adjacent and share $\{x, y\}$;
they differ only in w (only u) and z (only v) $\Rightarrow$ near-twins.

Prior Idea 1 (cont.): Near-Twins

Definition

Two vertices u, v are **near-twins** if they are *adjacent* and their neighborhoods are *nearly identical* (differ in only a few vertices).

- Intuition: twin classes are redundancy hotspots; covering them early should pay off.



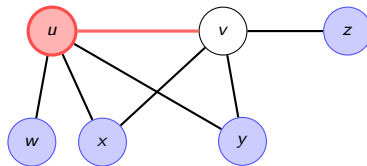
u, v are adjacent and share $\{x, y\}$;
they differ only in w (only u) and z (only v) $\Rightarrow$ near-twins.

Prior Idea 1 (cont.): Near-Twins

Definition

Two vertices u, v are **near-twins** if they are *adjacent* and their neighborhoods are *nearly identical* (differ in only a few vertices).

- Intuition: twin classes are redundancy hotspots; covering them early should pay off.
- H5 *seeds* the solution with near-twin vertices, then continues greedily.

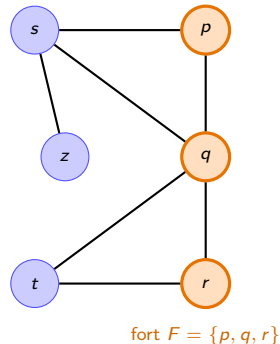


u, v are adjacent and share $\{x, y\}$;
they differ only in w (only u) and z (only v) $\Rightarrow$ near-twins.

Prior Idea 2: Forts

Definition (Smith and Hicks, 2022)

A non-empty $F \subseteq V$ is a **fort** if every vertex *outside* F has either **0** or ≥ 2 neighbors in F .



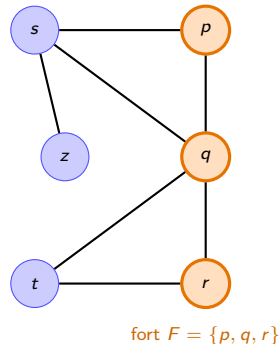
Outside vertices: s has 2 neighbors in F (p, q); t has 2 (q, r); z has 0. All satisfy the “0 or ≥ 2 ” rule, so F is a fort.

Prior Idea 2: Forts

Definition (Smith and Hicks, 2022)

A non-empty $F \subseteq V$ is a **fort** if every vertex *outside* F has either **0** or ≥ 2 neighbors in F .

- A fort is a “hard core” that propagation cannot break into from outside.



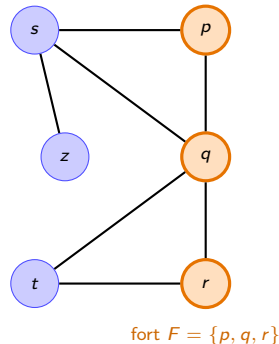
Outside vertices: s has 2 neighbors in F (p, q); t has 2 (q, r); z has 0. All satisfy the “0 or ≥ 2 ” rule, so F is a fort.

Prior Idea 2: Forts

Definition (Smith and Hicks, 2022)

A non-empty $F \subseteq V$ is a **fort** if every vertex *outside* F has either **0** or ≥ 2 neighbors in F .

- A fort is a “hard core” that propagation cannot break into from outside.
- Every PDS must dominate into each fort — forts are now the **modern foundation of exact solvers** for PDS.



Outside vertices: s has 2 neighbors in F (p, q); t has 2 (q, r); z has 0. All satisfy the “0 or ≥ 2 ” rule, so F is a fort.

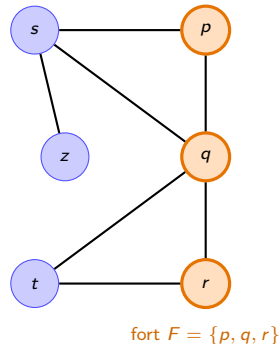
Prior Idea 2: Forts

Definition (Smith and Hicks, 2022)

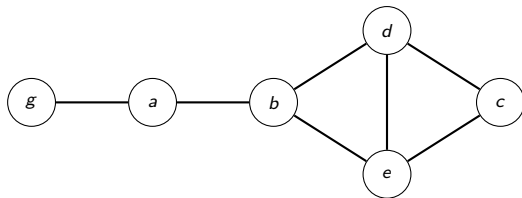
A non-empty $F \subseteq V$ is a **fort** if every vertex *outside* F has either **0** or ≥ 2 neighbors in F .

- A fort is a “hard core” that propagation cannot break into from outside.
- Every PDS must dominate into each fort — forts are now the **modern foundation of exact solvers** for PDS.
- We use forts for **pruning**.

Outside vertices: s has 2 neighbors in F (p, q); t has 2 (q, r); z has 0. All satisfy the “0 or ≥ 2 ” rule, so F is a fort.

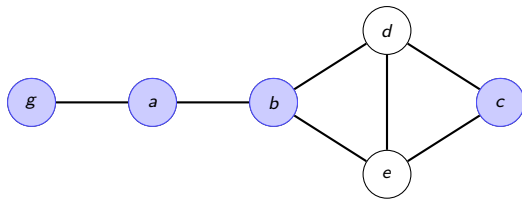


Why Propagation Cannot Enter a Fort



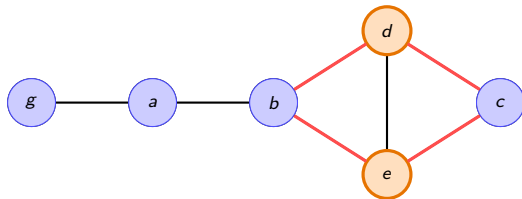
Fort $F = \{d, e\}$. Every outside vertex has 0 or ≥ 2 neighbors in F (b, c have 2; g, a have 0).

Why Propagation Cannot Enter a Fort



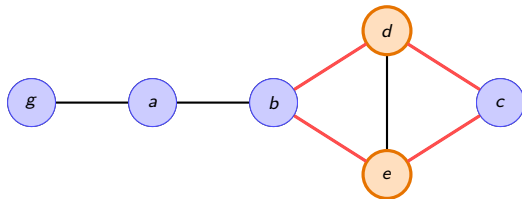
Suppose *everything outside F* is observed (blue), but *no* vertex of F is yet.

Why Propagation Cannot Enter a Fort



Try to propagate into F : b is observed but has *two* unobserved neighbors $\{d, e\}$ — the rule needs *exactly one*, so b cannot force. Same for c . (g, a have no neighbor in F .)

Why Propagation Cannot Enter a Fort



Conclusion: no observed vertex can force a vertex of F . The fort stays unobserved forever $\Rightarrow$ every PDS must place / dominate a PMU *inside* each fort.

Pruning: Naive vs. Fort-Based

Greedy solutions contain redundant PMUs. Pruning: test each $v \in S$ for removal; if $S \setminus \{v\}$ still observes everything, drop v .

Pruning: Naive vs. Fort-Based

Greedy solutions contain redundant PMUs. Pruning: test each $v \in S$ for removal; if $S \setminus \{v\}$ still observes everything, drop v .

Naive pruner

Re-runs the **full observation routine** ($O(m)$) for *every* candidate vertex.

Pruning: Naive vs. Fort-Based

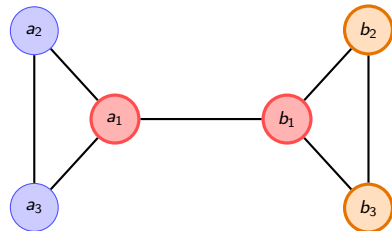
Greedy solutions contain redundant PMUs. Pruning: test each $v \in S$ for removal; if $S \setminus \{v\}$ still observes everything, drop v .

Naive pruner

Re-runs the **full observation routine** ($O(m)$) for every candidate vertex.

Fort-based pruner

- When a removal *fails*, the unobserved remainder is a **fort** — cache it.
- A vertex that is the *sole* PMU covering a cached fort is essential: **lock it** and skip the re-check.



$S = \{a_1, b_1\}$ (red). Removing b_1 leaves $\{b_2, b_3\}$ unobserved — a **fort**. b_1 is its only cover $\Rightarrow$ **lock** b_1 , cache the fort.

Pruning: Naive vs. Fort-Based

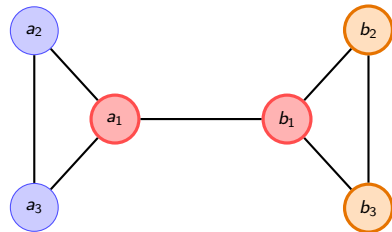
Greedy solutions contain redundant PMUs. Pruning: test each $v \in S$ for removal; if $S \setminus \{v\}$ still observes everything, drop v .

Naive pruner

Re-runs the **full observation routine** ($O(m)$) for every candidate vertex.

Fort-based pruner

- When a removal *fails*, the unobserved remainder is a **fort** — cache it.
- A vertex that is the *sole* PMU covering a cached fort is essential: **lock it** and skip the re-check.



$S = \{a_1, b_1\}$ (red). Removing b_1 leaves $\{b_2, b_3\}$ unobserved — a **fort**. b_1 is its only cover $\Rightarrow$ **lock** b_1 , cache the fort.

Both pruners reach **identical minimal sizes**; the fort-based pruner skips $\approx$ half the closure calls, paying off on the largest grids.

Experimental Setup

Benchmark: we run our heuristics on a range of standard power-grid benchmark datasets (BCSPWR, IEEE, PEGASE, RTE, ...),

$14 \leq n \leq 9,241$

- **BCSPWR01–10** (SuiteSparse / Harwell–Boeing power networks), $n = 39$ to 5300

Experimental Setup

Benchmark: we run our heuristics on a range of standard power-grid benchmark datasets (BCSPWR, IEEE, PEGASE, RTE, ...),

$14 \leq n \leq 9,241$

- **BCSPWR01–10** (SuiteSparse / Harwell–Boeing power networks), $n = 39$ to 5300
- **Bus admittance cases:** 494, 662, 685, 1138 bus

Experimental Setup

Benchmark: we run our heuristics on a range of standard power-grid benchmark datasets (BCSPWR, IEEE, PEGASE, RTE, ...),

$14 \leq n \leq 9,241$

- **BCSPWR01–10** (SuiteSparse / Harwell–Boeing power networks), $n = 39$ to 5300
- **Bus admittance cases:** 494, 662, 685, 1138 bus
- **IEEE** standard cases: 14, 30, 57, 118, 300 + Illinois 200

Experimental Setup

Benchmark: we run our heuristics on a range of standard power-grid benchmark datasets (BCSPWR, IEEE, PEGASE, RTE, ...),

$14 \leq n \leq 9,241$

- **BCSPWR01–10** (SuiteSparse / Harwell–Boeing power networks), $n = 39$ to 5300
- **Bus admittance cases:** 494, 662, 685, 1138 bus
- **IEEE** standard cases: 14, 30, 57, 118, 300 + Illinois 200
- **PEGASE** European cases: 1354, 2869, 9241

Experimental Setup

Benchmark: we run our heuristics on a range of standard power-grid benchmark datasets (BCSPWR, IEEE, PEGASE, RTE, ...),

$14 \leq n \leq 9,241$

- **BCSPWR01–10** (SuiteSparse / Harwell–Boeing power networks), $n = 39$ to 5300
- **Bus admittance cases:** 494, 662, 685, 1138 bus
- **IEEE** standard cases: 14, 30, 57, 118, 300 + Illinois 200
- **PEGASE** European cases: 1354, 2869, 9241
- **RTE** French grid cases: 1888, 1951, 2848, 2868, 6468, 6470

Experimental Setup

Benchmark: we run our heuristics on a range of standard power-grid benchmark datasets (BCSPWR, IEEE, PEGASE, RTE, ...),

$14 \leq n \leq 9,241$

- **BCSPWR01–10** (SuiteSparse / Harwell–Boeing power networks), $n = 39$ to 5300
- **Bus admittance cases:** 494, 662, 685, 1138 bus
- **IEEE** standard cases: 14, 30, 57, 118, 300 + Illinois 200
- **PEGASE** European cases: 1354, 2869, 9241
- **RTE** French grid cases: 1888, 1951, 2848, 2868, 6468, 6470

Protocol

- All five heuristics + both pruners on every instance; single-run greedy, semi-greedy (top- k), and multi-run semi-greedy

Experimental Setup

Benchmark: we run our heuristics on a range of standard power-grid benchmark datasets (BCSPWR, IEEE, PEGASE, RTE, ...),

$14 \leq n \leq 9,241$

- **BCSPWR01–10** (SuiteSparse / Harwell–Boeing power networks), $n = 39$ to 5300
- **Bus admittance cases:** 494, 662, 685, 1138 bus
- **IEEE** standard cases: 14, 30, 57, 118, 300 + Illinois 200
- **PEGASE** European cases: 1354, 2869, 9241
- **RTE** French grid cases: 1888, 1951, 2848, 2868, 6468, 6470

Protocol

- All five heuristics + both pruners on every instance; single-run greedy, semi-greedy (top- k), and multi-run semi-greedy
- Metrics: solution size $|S|$, wall-clock time, closure calls, and

$$\%Dev = 100 \cdot \frac{|S| - \gamma_P}{\gamma_P}$$

against certified optima of Bläsius and Göttlicher (2025)

Construction Quality Varies Over the Heuristics

Instance	n	m	H1	H2	H3	H4	H5
BCSPWR01	39	46	14	6	16	8	10
IEEE57	57	78	12	4	14	6	12
IEEE300	300	409	155	36	245	40	164
1138_bus	1138	1458	434	149	878	170	480
pegase1354	1354	1710	495	180	1121	213	550
case1888rte	1888	2308	582	254	1363	291	778
BCSPWR10	5300	8271	2275	366	4465	442	2307
case6470rte	6470	8066	2146	802	5582	911	2511
pegase9241	9241	14207	3798	920	8119	1058	3975

Raw (un-pruned) solution sizes, single-run greedy; selected instances. Best per row in bold.

- On BCSPWR10, the solution size ranges from 366 (H2) up to 4465 (H3) — a factor of about $12\times$.

Construction Quality Varies Over the Heuristics

Instance	n	m	H1	H2	H3	H4	H5
BCSPWR01	39	46	14	6	16	8	10
IEEE57	57	78	12	4	14	6	12
IEEE300	300	409	155	36	245	40	164
1138_bus	1138	1458	434	149	878	170	480
pegase1354	1354	1710	495	180	1121	213	550
case1888rte	1888	2308	582	254	1363	291	778
BCSPWR10	5300	8271	2275	366	4465	442	2307
case6470rte	6470	8066	2146	802	5582	911	2511
pegase9241	9241	14207	3798	920	8119	1058	3975

Raw (un-pruned) solution sizes, single-run greedy; selected instances. Best per row in bold.

- On BCSPWR10, the solution size ranges from 366 (H2) up to 4465 (H3) — a factor of about $12\times$.
- Multi-hop (H3) and twin-guided (H5) over-reward high-degree hubs and structural redundancy.

Construction Quality Varies Over the Heuristics

Instance	n	m	H1	H2	H3	H4	H5
BCSPWR01	39	46	14	6	16	8	10
IEEE57	57	78	12	4	14	6	12
IEEE300	300	409	155	36	245	40	164
1138_bus	1138	1458	434	149	878	170	480
pegase1354	1354	1710	495	180	1121	213	550
case1888rte	1888	2308	582	254	1363	291	778
BCSPWR10	5300	8271	2275	366	4465	442	2307
case6470rte	6470	8066	2146	802	5582	911	2511
pegase9241	9241	14207	3798	920	8119	1058	3975

Raw (un-pruned) solution sizes, single-run greedy; selected instances. Best per row in bold.

- On BCSPWR10, the solution size ranges from 366 (H2) up to 4465 (H3) — a factor of about $12\times$.
- Multi-hop (H3) and twin-guided (H5) over-reward high-degree hubs and structural redundancy.
- Graphs richer in near-twin structure should benefit more from H5 — on such graphs the picture may differ.

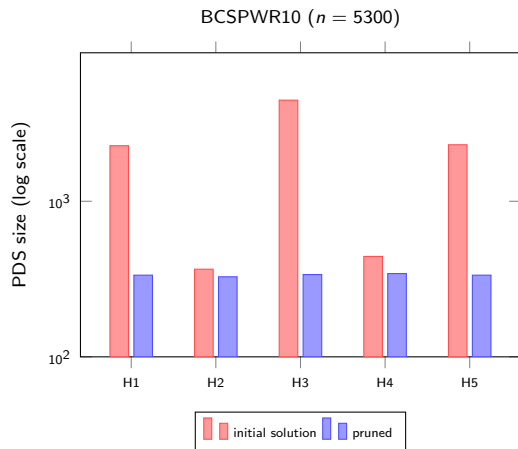
Construction Quality Varies Over the Heuristics

Instance	n	m	H1	H2	H3	H4	H5
BCSPWR01	39	46	14	6	16	8	10
IEEE57	57	78	12	4	14	6	12
IEEE300	300	409	155	36	245	40	164
1138_bus	1138	1458	434	149	878	170	480
pegase1354	1354	1710	495	180	1121	213	550
case1888rte	1888	2308	582	254	1363	291	778
BCSPWR10	5300	8271	2275	366	4465	442	2307
case6470rte	6470	8066	2146	802	5582	911	2511
pegase9241	9241	14207	3798	920	8119	1058	3975

Raw (un-pruned) solution sizes, single-run greedy; selected instances. Best per row in bold.

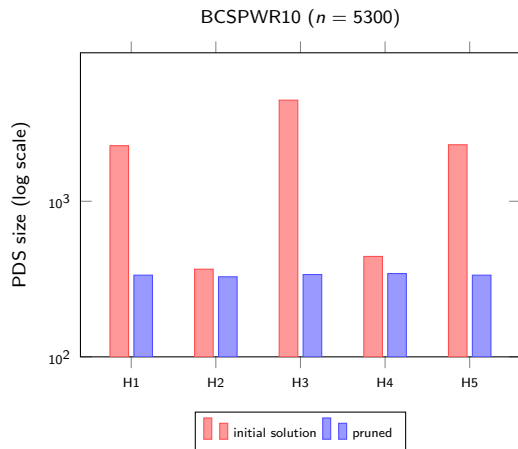
- On BCSPWR10, the solution size ranges from 366 (H2) up to 4465 (H3) — a factor of about $12\times$.
- Multi-hop (H3) and twin-guided (H5) over-reward high-degree hubs and structural redundancy.
- Graphs richer in near-twin structure should benefit more from H5 — on such graphs the picture may differ.
- **Effective-degree greedy (H2) introduces the least redundancy throughout.**

Result 2: Pruning is the Great Equalizer



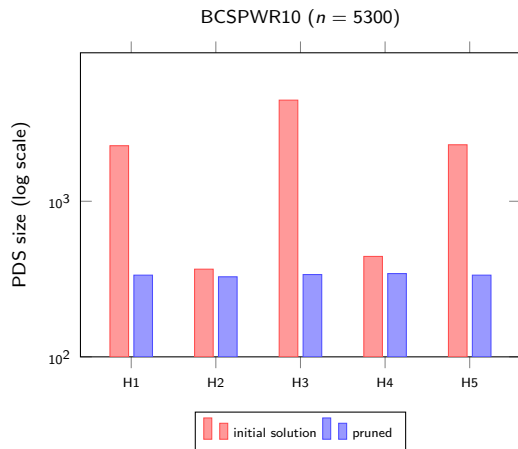
- After pruning, all five constructors converge to a **narrow band**: on BCSPWR10, 327–343 (spread < 5%) despite initial solution sizes differing by $12\times$.

Result 2: Pruning is the Great Equalizer



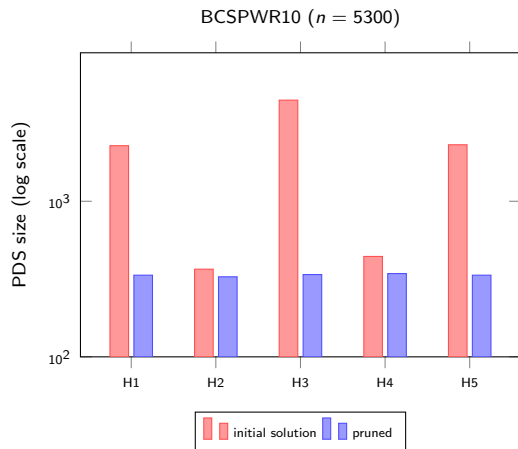
- After pruning, all five constructors converge to a **narrow band**: on BCSPWR10, 327–343 (spread < 5%) despite initial solution sizes differing by $12\times$.
- This pattern holds across the whole benchmark.

Result 2: Pruning is the Great Equalizer



- After pruning, all five constructors converge to a **narrow band**: on BCSPWR10, 327–343 (spread < 5%) despite initial solution sizes differing by $12\times$.
- This pattern holds across the whole benchmark.
- **The pruning stage, rather than the construction strategy, determines the final solution size.**

Result 2: Pruning is the Great Equalizer



- After pruning, all five constructors converge to a **narrow band**: on BCSPWR10, 327–343 (spread < 5%) despite initial solution sizes differing by $12\times$.
- This pattern holds across the whole benchmark.
- **The pruning stage, rather than the construction strategy, determines the final solution size.**
- Still, H2 yields the smallest pruned set on most graphs (e.g. 327 on BCSPWR10, the minimum across all methods).

Result 2 (cont.): Raw vs. Pruned Across the Benchmark

Instance	n	H1	H2	H3	H4	H5
BCSPWR01	39	14/5	6/ 5	16/6	8/ 5	10/ 5
IEEE118	118	36/10	10/9	70/10	11/ 8	41/10
illinois200	200	44/21	23/ 20	98/29	25/ 20	46/21
685_bus	685	276/50	50/45	580/61	59/ 41	284/50
pegase1354	1354	495/182	180/ 177	1121/251	213/178	550/181
case1888rte	1888	582/246	254/ 238	1363/359	291/240	778/264
case2848rte	2848	884/364	381/ 355	2229/559	439/ 355	1139/404
BCSPWR10	5300	2275/335	366/ 327	4465/338	442/343	2307/335
case6470rte	6470	2146/802	802/ 754	5582/1228	911/ 754	2511/841
pegase9241	9241	3798/897	920/ 850	8119/1225	1058/859	3975/918

Entries are raw / pruned solution sizes (naive and fort-based pruning give identical sizes). Best pruned size per row in bold.

Optimality Gap

Instance	γ_P	H1	H2	H3	H4	H5
IEEE118	8	25.00	12.50	25.00	0.00	25.00
illinois200	20	5.00	0.00	45.00	0.00	5.00
pegase1354	176	3.41	0.57	42.61	1.14	2.84
case1888rte	235	4.68	1.28	52.77	2.13	12.34
case2848rte	352	3.41	0.85	58.81	0.85	14.77
case6470rte	745	7.65	1.21	64.83	1.21	12.89
pegase9241	811	10.60	4.81	51.05	5.92	13.19
Average		8.54	3.03	48.58	1.61	12.29

%Dev of greedy + pruning vs. certified γ_P from Bläsius and Göttlicher (2025), Table 2 (pandapower dataset).

- **H2 stays closest to the optimum** on every instance; H3 is far off.

Result 4: Fort-Based Pruning Pays Off on Large Grids

Instance	Naive (s)	Fort (s)	Speedup
BCSPWR06	1.03	0.51	2.01×
BCSPWR07	1.13	0.60	1.87×
BCSPWR08	1.33	0.67	1.99×
BCSPWR09	1.59	0.89	1.79×
BCSPWR10	10.24	5.56	1.84×
pegase1354	0.60	0.35	1.75×
case1951rte	1.74	0.75	2.34×
pegase2869	3.47	1.74	1.99×
case6468rte	17.54	9.02	1.94×

Pruning time for H2 solutions, naive vs. fort-based.

- Both pruners reach **identical minimal sizes** everywhere.

Result 4: Fort-Based Pruning Pays Off on Large Grids

Instance	Naive (s)	Fort (s)	Speedup
BCSPWR06	1.03	0.51	2.01×
BCSPWR07	1.13	0.60	1.87×
BCSPWR08	1.33	0.67	1.99×
BCSPWR09	1.59	0.89	1.79×
BCSPWR10	10.24	5.56	1.84×
pegase1354	0.60	0.35	1.75×
case1951rte	1.74	0.75	2.34×
pegase2869	3.47	1.74	1.99×
case6468rte	17.54	9.02	1.94×

Pruning time for H2 solutions, naive vs. fort-based.

- Both pruners reach **identical minimal sizes** everywhere.
- For H2 the fort-based pruner is consistently $\approx 1.8\text{--}2\times$ **faster** on the larger grids.

Result 5: Semi-Greedy Randomization — Single Pass

Semi-greedy: at each step pick uniformly among the top- k scored candidates.

Instance (raw, H2)	$k = 1$	$k = 3$	$k = 5$	$k = 10$
BCSPWR01	6	5	6	7
IEEE57	4	6	5	6
IEEE118	10	10	12	12
IEEE300	36	36	37	39
illinois200	23	21	36	44
pegase1354	180	181	189	194
BCSPWR10	366	367	362	370
case6470rte	802	810	804	818
pegase9241	920	933	934	934

Raw H2 solution sizes for a single semi-greedy pass at different k ($k = 1$ is pure greedy). Best per row in bold.

Result 5: Semi-Greedy Randomization — Single Pass

Semi-greedy: at each step pick uniformly among the top- k scored candidates.

Instance (raw, H2)	$k = 1$	$k = 3$	$k = 5$	$k = 10$
BCSPWR01	6	5	6	7
IEEE57	4	6	5	6
IEEE118	10	10	12	12
IEEE300	36	36	37	39
illinois200	23	21	36	44
pegase1354	180	181	189	194
BCSPWR10	366	367	362	370
case6470rte	802	810	804	818
pegase9241	920	933	934	934

Raw H2 solution sizes for a single semi-greedy pass at different k ($k = 1$ is pure greedy). Best per row in bold.

- For **small** k ($k = 3$), semi-greedy works well: it stays close to pure greedy and sometimes improves on it (BCSPWR01: $6 \rightarrow 5$; illinois200: $23 \rightarrow 21$).

Result 5: Semi-Greedy Randomization — Single Pass

Semi-greedy: at each step pick uniformly among the top- k scored candidates.

Instance (raw, H2)	$k = 1$	$k = 3$	$k = 5$	$k = 10$
BCSPWR01	6	5	6	7
IEEE57	4	6	5	6
IEEE118	10	10	12	12
IEEE300	36	36	37	39
illinois200	23	21	36	44
pegase1354	180	181	189	194
BCSPWR10	366	367	362	370
case6470rte	802	810	804	818
pegase9241	920	933	934	934

Raw H2 solution sizes for a single semi-greedy pass at different k ($k = 1$ is pure greedy). Best per row in bold.

- For **small** k ($k = 3$), semi-greedy works well: it stays close to pure greedy and sometimes improves on it (BCSPWR01: $6 \rightarrow 5$; illinois200: $23 \rightarrow 21$).
- For **large** k ($k = 10$) we got no advantage: too much randomization dilutes the score signal (illinois200: $23 \rightarrow 44$).

Result 5 (cont.): Multiple Runs Recover Small Gains

Multi-run semi-greedy: keep the **best** of several randomized passes.

Instance	H2 single ($k = 1$)	H2 multi-run best	Improvement
BCSPWR01	6	5	1
IEEE57	4	3	1
IEEE118	10	9	1
IEEE300	36	34	2
illinois200	23	22	1
494_bus	63	61	2
pegase1354	180	178	2
pegase2869	326	326	0
pegase9241	920	917	3
case6470rte	802	802	0

Raw H2 sizes: single pure-greedy pass vs. best of multiple semi-greedy runs.

Result 5 (cont.): Multiple Runs Recover Small Gains

Multi-run semi-greedy: keep the **best** of several randomized passes.

Instance	H2 single ($k = 1$)	H2 multi-run best	Improvement
BCSPWR01	6	5	1
IEEE57	4	3	1
IEEE118	10	9	1
IEEE300	36	34	2
illinois200	23	22	1
494_bus	63	61	2
pegase1354	180	178	2
pegase2869	326	326	0
pegase9241	920	917	3
case6470rte	802	802	0

Raw H2 sizes: single pure-greedy pass vs. best of multiple semi-greedy runs.

- Multiple runs recover **small** improvements on the raw construction (also e.g. IEEE14: H1 $4 \rightarrow 2$).

Effect of Pruning

Instance	Greedy	Semi-greedy ($k=3$)	Multi-run
IEEE57	4	5	3
IEEE118	9	8	8
IEEE300	31	31	33
illinois200	20	20	20
494_bus	55	58	55
BCSPWR05	48	48	47
1138_bus	139	139	139
pegase1354	177	177	177
pegase2869	309	309	310
case6468rte	736	738	737

Pruned H2 sizes for the three construction strategies (naive and fort-based pruning give identical sizes). Best per row in bold.

- After pruning, plain greedy, semi-greedy ($k = 3$) and multi-run all land within **1–2 PMUs** of each other on every instance.

Effect of Pruning

Instance	Greedy	Semi-greedy ($k=3$)	Multi-run
IEEE57	4	5	3
IEEE118	9	8	8
IEEE300	31	31	33
illinois200	20	20	20
494_bus	55	58	55
BCSPWR05	48	48	47
1138_bus	139	139	139
pegase1354	177	177	177
pegase2869	309	309	310
case6468rte	736	738	737

Pruned H2 sizes for the three construction strategies (naive and fort-based pruning give identical sizes). Best per row in bold.

- After pruning, plain greedy, semi-greedy ($k = 3$) and multi-run all land within **1–2 PMUs** of each other on every instance.
- **The pruning stage equalizes the construction strategies** — the extra runs buy almost nothing once pruning is applied.

Effect of Pruning

Instance	Greedy	Semi-greedy ($k=3$)	Multi-run
IEEE57	4	5	3
IEEE118	9	8	8
IEEE300	31	31	33
illinois200	20	20	20
494_bus	55	58	55
BCSPWR05	48	48	47
1138_bus	139	139	139
pegase1354	177	177	177
pegase2869	309	309	310
case6468rte	736	738	737

Pruned H2 sizes for the three construction strategies (naive and fort-based pruning give identical sizes). Best per row in bold.

- After pruning, plain greedy, semi-greedy ($k = 3$) and multi-run all land within **1–2 PMUs** of each other on every instance.
- **The pruning stage equalizes the construction strategies** — the extra runs buy almost nothing once pruning is applied.
- **Takeaway:** for PDS, randomized construction needs a strong improvement phase (e.g. path relinking) to be worthwhile.

Key Findings

- ➊ **H2 (effective-degree greedy) works best:** it builds the smallest solutions among all the heuristics.

Key Findings

- ① **H2 (effective-degree greedy) works best:** it builds the smallest solutions among all the heuristics.
- ② **Pruning greatly reduces the solution size,** and brings all the heuristics down to nearly the same size.

Key Findings

- ① **H2 (effective-degree greedy) works best:** it builds the smallest solutions among all the heuristics.
- ② **Pruning greatly reduces the solution size,** and brings all the heuristics down to nearly the same size.
- ③ **H2 together with pruning gives solutions very close to the optimum** on real power grids.

Key Findings

- ① **H2 (effective-degree greedy) works best:** it builds the smallest solutions among all the heuristics.
- ② **Pruning greatly reduces the solution size,** and brings all the heuristics down to nearly the same size.
- ③ **H2 together with pruning gives solutions very close to the optimum** on real power grids.
- ④ **Fort-based pruning works well:** it reaches the same sizes as the naive pruner while saving about half of our running time on the large grids.

Future Work

- Local search / path relinking on top of (semi-)greedy construction.

Future Work

- Local search / path relinking on top of (semi-)greedy construction.
- Grid-specific reduction rules that exploit power-network topology directly.

References

- [1] T. W. Haynes et al. Domination in graphs applied to electric power networks. *SIAM J. Discrete Math.*, 15(4):519–529, 2002.
- [2] D. J. Brueni and L. S. Heath. The PMU placement problem. *SIAM J. Discrete Math.*, 19(3):744–761, 2005.
- [3] A. Aazami. Hardness results and approximation algorithms for some problems on graphs. PhD thesis, 2008.
- [4] A. Aazami and M. D. Stilp. Approximation algorithms and hardness for domination with propagation. *SIAM J. Discrete Math.*, 23(3):1382–1399, 2009.
- [5] D. Binkele-Raible and H. Fernau. An exact exponential time algorithm for power dominating set. *Algorithmica*, 63(1):323–346, 2012.
- [6] R. Jovanovic and S. Voss. Greedy/GRASP heuristics for the power dominating set problem, 2020.
- [7] L. Smith and I. Hicks. Optimal sensor placement in power grids: fort-based branch-and-cut, 2022.
- [8] T. Bläsius and M. Göttlicher. An efficient algorithm for power dominating set. *Algorithmica*, 87(3):344–376, 2025.
- [9] M. Laguna. Multiparent path relinking: an application to the power dominating set problem. *Math. Prog. Comp.*, 17:759–779, 2025.

Thank You

Questions?